

250508 : 개인연구미팅 준비로 인해 Term Project 진행 내용 X

250509 : 잔차 기준 업데이트

각 솔버의 수렴 기준을 기존에는 최대 온도 변화량이 tolerance 이하이면 수렴한다고 판단하여 코드를 작성했지만, 오늘 수업을 듣고 와서 residual 방식의 수렴 방식을 써야함을 배우게 되었다. 따라서 지금 풀고 있는 온도장 문제에서는 energy conservation equation의 수렴 기준을 정하여, 수렴했음을 판단해야 한다.

fully developed 된 상황이므로, $\epsilon = 0.001$ 로 두어 잔차도 수렴 기준에 포함하고자 한다.

1. 현재 코드의 수렴 기준

- $\max|T_{\text{new}} - T_{\text{old}}| < \text{tol}$

→ “해의 변화량”이 충분히 작아지면 수렴으로 간주

장점

- 구현이 간단하고 직관적
- 해가 더 이상 변하지 않으면 “수렴”으로 판단

단점

- 해가 느리게 변해도(진동, 발산 직전 등) 수렴으로 오판할 수 있음
- 실제로 방정식(연립방정식)의 “잔차”가 충분히 작아졌는지 보장하지 않음

<기존 방법>

tol = 1e-6; % 수렴 허용 오차 (tolerance)

maxit = 5000; % 최대 반복 횟수

%SOR%

function T = sorSolve(coef, T, omega, tol, maxit)

...

for it=1:maxit

maxd=0;

...

if maxd<tol, return; end

end

warning('SOR maxit reached'); end

<잔차 수렴 기준 추가>

$$\text{Residual (Energy)} = \frac{\sum |a_P T_P - \sum a_{nb} T_{nb} - b|}{\sum \dot{m} c_p T_{in} + \sum \dot{m} c_p |T_w - T_{in}|}$$

```
% Residual 계산
residual = zeros(Ny,Nx);
for j = 2:Ny-1
    for i = 2:Nx
        residual(j,i) = abs( ...
            coef.aP(j,i)*T(j,i) ...
            - coef.aE(j,i)*T(j,min(i+1,Nx)) ...
            - coef.aW(j,i)*T(j,i-1) ...
            - coef.aN(j,i)*T(j+1,i) ...
            - coef.aS(j,i)*T(j-1,i) ...
            - coef.b(j,i) );
    end
end
res_sum = sum(residual(:));
m_dot   = rho * (2*1) * Uave;
escale  = m_dot * cp * Tin + m_dot * cp * abs(Tw - Tin);
norm_res= res_sum / escale;
```

따라서 두 수렴 기준이 모두 만족하는 경우에만 계산을 끝내는 것으로 판단하였다.

아래는 PowerLaw + SOR 솔버를 사용했을 때의 출력 결과이다.

```
SOR   iter  185: Max difference = 5.14e-06, Residual = 7.40e-11
SOR   iter  186: Max difference = 4.32e-06, Residual = 6.12e-11
SOR   iter  187: Max difference = 3.62e-06, Residual = 5.06e-11
SOR   iter  188: Max difference = 3.03e-06, Residual = 4.17e-11
SOR   iter  189: Max difference = 2.53e-06, Residual = 3.43e-11
SOR   iter  190: Max difference = 2.11e-06, Residual = 2.82e-11
SOR   iter  191: Max difference = 1.76e-06, Residual = 2.31e-11
SOR   iter  192: Max difference = 1.46e-06, Residual = 1.89e-11
SOR   iter  193: Max difference = 1.21e-06, Residual = 1.55e-11
SOR   iter  194: Max difference = 9.99e-07, Residual = 1.26e-11
SOR converged at iter 194
```

250510 : Grid Test 진행하여 적절한 grid size 선정

격자 테스트 진행을 위해, 테스트용 코드만 따로 파일로 만들었다.

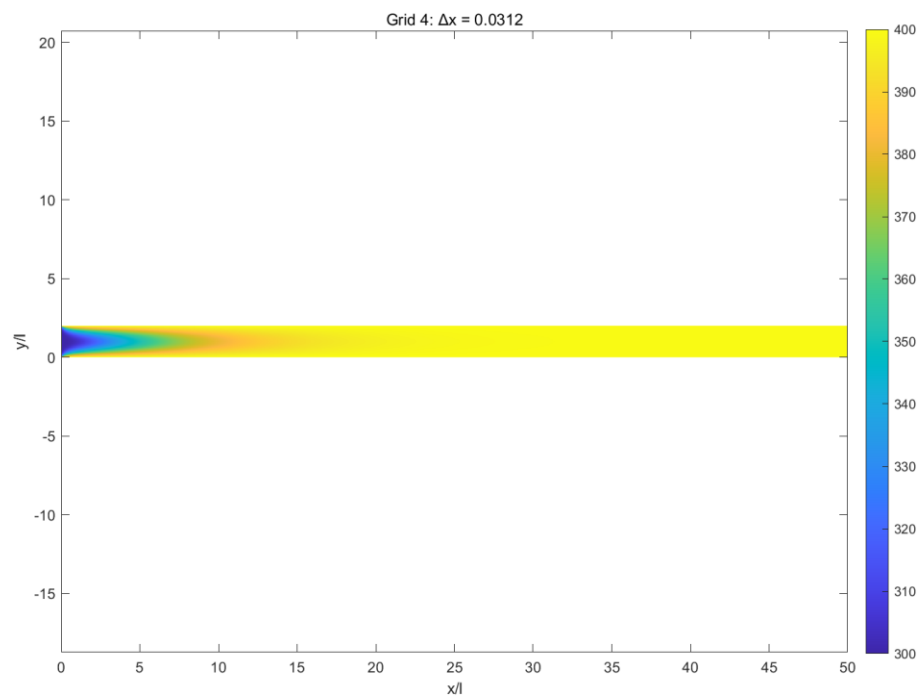
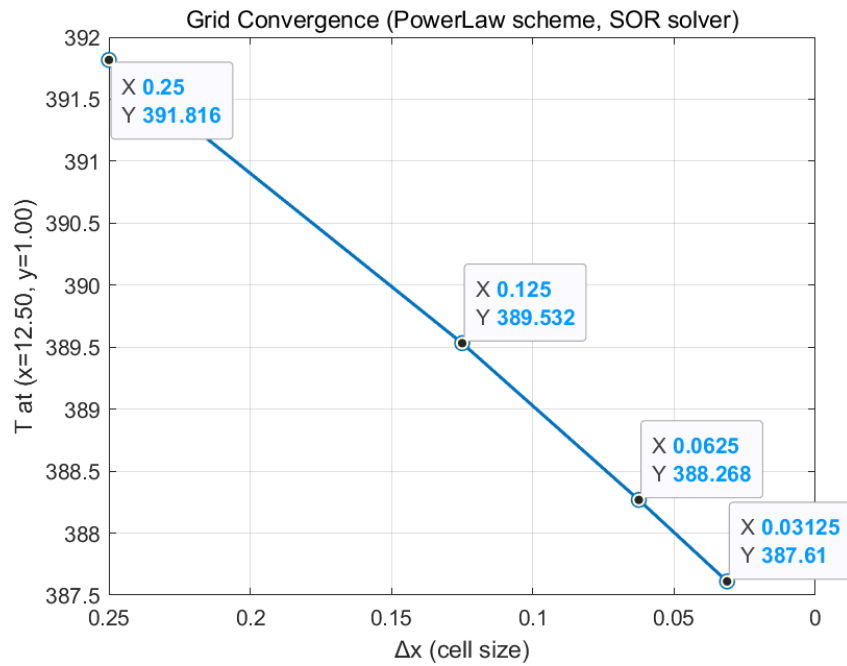
초기 격자 사이즈를 0.25로 두고 2배씩 줄여 0.25, 0.125, 0.0625, 0.03125 사이즈들에 대해 테스트를 진행했다.

수렴 테스트를 할 관심 위치는 x축 기준으로 50 l * 0.25에 해당하는 위치로 정하였다. (l = 1)

각 사이즈마다 0.25 l 에 해당하는 위치를 찾아서 온도를 저장했다.

```
128 %% --- Grid Convergence Test ---
129 divisions = [1,2,4,8]; % cell_size를 이들로 나눔
130 cellSizes = cell_size ./ divisions; % Δx 목록
131 Ndiv = numel(divisions);
132
133 monitorTemp = zeros(Ndiv,1);
134 x_monitor = Lx*0.25; % 모니터링 위치 (x,y)
135 y_monitor = Ly*0.50;
136
137 for k = 1:Ndiv
138     dx = cellSizes(k); dy = dx;
139     Nx = round(Lx/dx); Ny = round(Ly/dy);
140
141     % 셀 중심 & face 좌표
142     x_cent_g = dx/2:dx:Lx-dx/2;
143     y_cent_g = dy/2:dy:Ly-dy/2;
144     half_cx = dx/2; half_cy = dy/2;
145     dx_wg = half_cx*ones(1,Nx); dx_wg(1)=0;
146     dx_eg = half_cx*ones(1,Nx); dx_eg(end)=0;
147     dy_sg = half_cy*ones(1,Ny); dy_sg(1)=0;
148     dy_ng = half_cy*ones(1,Ny); dy_ng(end)=0;
149     x_faces_g = [x_cent_g - dx_wg, x_cent_g(end)+dx_eg(end)];
150     y_faces_g = [y_cent_g - dy_sg, y_cent_g(end)+dy_ng(end)];
151
152     % 속도 프로파일
153     y_rel_g = (y_cent_g - 1)/1;
154     u_prof_g = Umax*(1 - y_rel_g.^2);
155
156     % coefficient assembly
157     Afunc = Afuncs.(schemeSel);
158     aE = zeros(Ny,Nx); aW=aE; aN=aE; aS=aE; aP=aE; b=zeros(Ny,Nx);
159     for j = 1:Ny
160         for i = 1:Nx
161             Ae = y_faces_g(j+1)-y_faces_g(j); Aw = Ae;
162             An = x_faces_g(i+1)-x_faces_g(i); As = An;
163             de=dx_eg(i); dw=dx_wg(i); dn=dy_ng(j); ds=dy_sg(j);
164             Fe=rho*u_prof_g(j)*Ae; Fw=Fe; Fn=0; Fs=0;
165             De=(de>0)*Gamma*Ae/de; Dw=(dw>0)*Gamma*Aw/dw;
166             Dn=(dn>0)*Gamma*An/dn; Ds=(ds>0)*Gamma*As/ds;
167             if i==Nx, ae=0; else Pe=Fe/De; ae=De*Afunc(Pe)+max(-Fe,0); end
168             if j==1, aw=0; else Pw=Fw/Dw; aw=Dw*Afunc(Pw)+max(Fw,0); end
169             if j==Ny, an=0; else Pn=Fn/Dn; an=Dn*Afunc(Pn)+max(-Fn,0); end
170             if j==1, as=0; else Ps=Fs/Ds; as=Ds*Afunc(Ps)+max(Fs,0); end
171             aE(j,i)=ae; aW(j,i)=aw; aN(j,i)=an; aS(j,i)=as;
172             aP(j,i)=ae+aw+an+as + (Fe - Fw + Fn - Fs);
173         end
174     end
175
176     coef_g=struct('aE',aE,'aW',aW,'aN',aN,'aS',aS,'aP',aP,'b',b);
177
178     % 초기장
179     T0_g=Tin*ones(Ny,Nx);
180     T0_g(:,1)=Tin; T0_g(1,:)=Tw; T0_g(Ny,:)=Tw;
181
182     % 풀이
183     switch solverSel
184     case 'Jacobi', T_g=jacobiSolve(coef_g,T0_g,tol_diff,tol_resid,maxit);
185     case 'GS', T_g=gsSolve(coef_g,T0_g,tol_diff,tol_resid,maxit);
186     case 'SOR', T_g=sorSolve(coef_g,T0_g,omega,tol_diff,tol_resid,maxit);
187     case 'EJacobi', T_g=extJacobiSolve(coef_g,T0_g,alpha,tol_diff,tol_resid,maxit);
188     case 'LineByLine', T_g=lineByLineSolve(coef_g,T0_g,tol_diff,tol_resid,maxit);
189     case 'ADI', T_g=adiSolve(coef_g,T0_g,tol_diff,tol_resid,maxit);
190     end
191
192     % 모니터링 위치 인덱스 → 온도 저장
193     [~, ix] = min(abs(x_cent_g - x_monitor));
194     [~, jy] = min(abs(y_cent_g - y_monitor));
195     monitorTemp(k) = T_g(jy,ix);
196
197     figure; % 조건 없이 매 반복마다 새 창
198     imagesc(x_cent_g/1, y_cent_g/1, T_g);
199     axis xy equal; colorbar;
200     xlabel('x/l'); ylabel('y/l');
201     title(sprintf('Grid %d: Δx = %.4f', k, cellSizes(k)));
202 end
```

대류 스킴은 PoweLaw를 사용했고, SOR 솔버를 사용했다. (underrelaxation, alpha = 0.8)



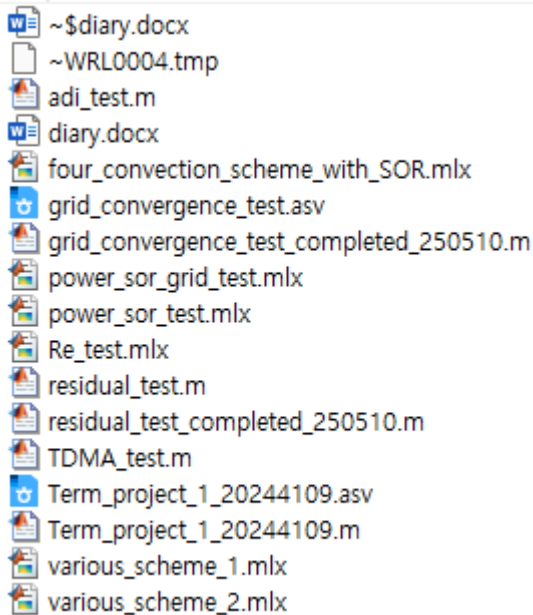
Division	dx	T_monitor	RelErr_to_next(%)	RelErr_to_fine(%)
1	0.2500	391.816	0.59	1.09
2	0.1250	389.532	0.33	0.50
4	0.0625	388.268	0.17	0.17
8	0.0312	387.610	N/A	0.00

격자 사이즈는 0.0625 정도만 써도 충분하다고 판단했고, 과제 제출용으로는 0.0312를 쓰려고 한다.

현재까지 작성 중인 코드는 아래와 같다.

문제 ©번까지 답할 수 있는 코드를 만들었다.

- (a) Determine the appropriate boundary conditions.
- (b) Obtain the discretization equations.
- (c) Plot the temperature distributions at the Reynolds number of 50.
- (d) Compare the results for the Nusselt number development with available data.



250511 : Nusselt number development

평판 사이에서의 Nusselt number development를 보기 위해 아래 논문을 참고했다.



PERGAMON

International Journal of Heat and Mass Transfer 45 (2002) 3263–3266

International Journal of
**HEAT and MASS
TRANSFER**

www.elsevier.com/locate/ijhmt

Technical Note

Improved approximation for the Nusselt number for hydrodynamically developed laminar flow between parallel plates

Markus Nickolay ^{a,*}, Holger Martin ^b

^a API Schmidt-Bretten GmbH & Co. KG, Pforzheimer Strasse 46, D-75015 Bretten, Germany

^b Thermische Verfahrenstechnik, Universität Karlsruhe (TH), D-76128 Karlsruhe, Germany

Received 14 December 2000; received in revised form 11 July 2001

위 논문 저자들이 발표한 approximation은 추가적인 계산이 소요되므로, 논문에서 참고하고 있는 기존의 방법인 Shah and London의 piecewise defined function을 사용하고자 한다.

R.K. Shah, A.L. London, Advances in heat transfer, in: T.F. Irvine Jr., J.P. Hartnett (Eds.), Advances in Heat Transfer (Suppl. 1), Academic Press, New York, 1978

Shah and London [3] propose the piecewise defined function

$$Nu_{SL} = \begin{cases} Nu_1 + 0.0235Gz & Gz < 166.\bar{6} \\ Nu_2 + 0.6 & 166.\bar{6} \geq Gz < 2000 \\ Nu_2 & Gz \geq 2000 \end{cases} \quad (3)$$

with

$$Nu_1 = \lim_{Gz \rightarrow 0} (Nu) = 7.541 \quad \text{and} \quad Nu_2 = \lim_{Gz \rightarrow \infty} (Nu) = 1.849Gz^{1/3} \quad (4)$$

for the mean Nusselt number and

$$Nu_{SL,x} = \begin{cases} Nu_1 + 6.874(Gz/1000)^{0.488} \exp\left(-\frac{245}{Gz}\right) & Gz < 1000 \\ 2/3Nu_2 + 0.4 & Gz \geq 1000 \end{cases} \quad (5)$$

for the local Nusselt number.

먼저 수치해석적으로 국소 Nusselt number를 구하기 위해, 평균 온도를 계산했다. 유속이 빠른 곳이 평균온도에 더 큰 영향을 미친다.

$$T_{\text{bulk}}(x_i) = \frac{\int_0^{2l} u(y) T(x_i, y) dy}{\int_0^{2l} u(y) dy} \quad h_x = -k \frac{\frac{\partial T}{\partial y} \Big|_{\text{wall}}}{T_w - T_{\text{bulk}}}$$

$$Nu_x = \frac{h_x D_h}{k}$$

```

133 %% --- 국소 및 평균 Nusselt 수 계산 ---
134 Pr = cp * mu / k; % Prandtl 수
135 Dh = 4*1; % hydraulic diameter (4A/P = 4*(21*1)/(2*1) )
136 % 1) 국소 Nusselt 수 Nu_loc 계산
137 Nu_loc = zeros(1, Nx);
138 T_bulk = zeros(1, Nx);
139 for i = 1:Nx
140     % 덩어리 평균 온도
141     T_bulk(i) = trapz(y_cent, u_prof .* T(:,i)) / trapz(y_cent, u_prof);
142     % 벽면 온도 기울기 근사 (첫 번째 셀 중심 사용)
143     dTdy_wall = (T(2,i) - Tw) / (y_cent(2) - y_cent(1));
144     h_x = -k * dTdy_wall / (Tw - T_bulk(i));
145     Nu_loc(i) = h_x * Dh / k;
146 end
147
148 % 2) 수치 해석 평균 Nu
149 Nu_avg_num = trapz(x_cent, Nu_loc) / Lx;
150

```

수치해석 해로 구한 local Nusselt number는 위와 같이 계산하였다.

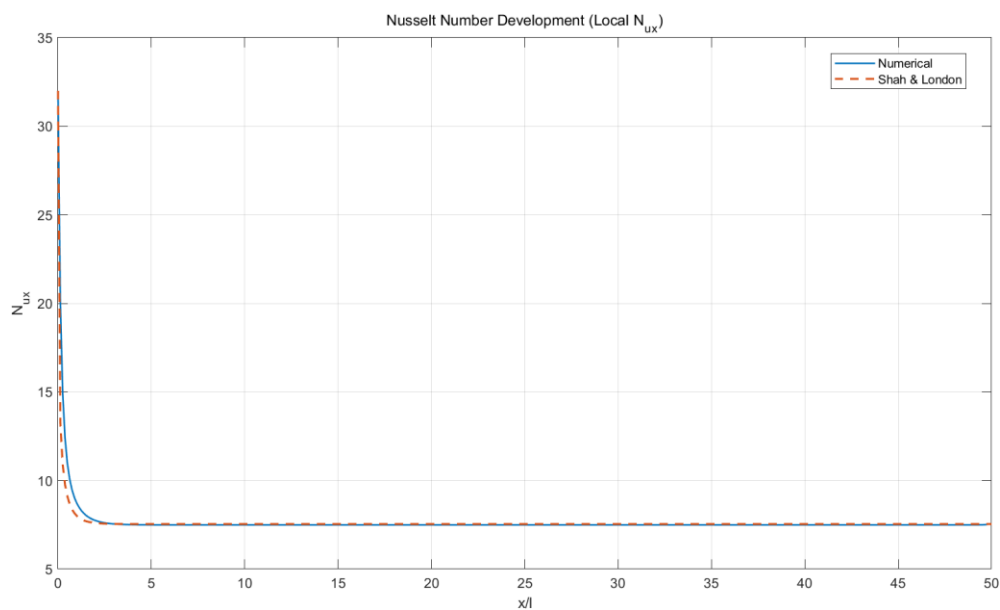
다음으로 논문에 있던 식을 그대로 사용했다.

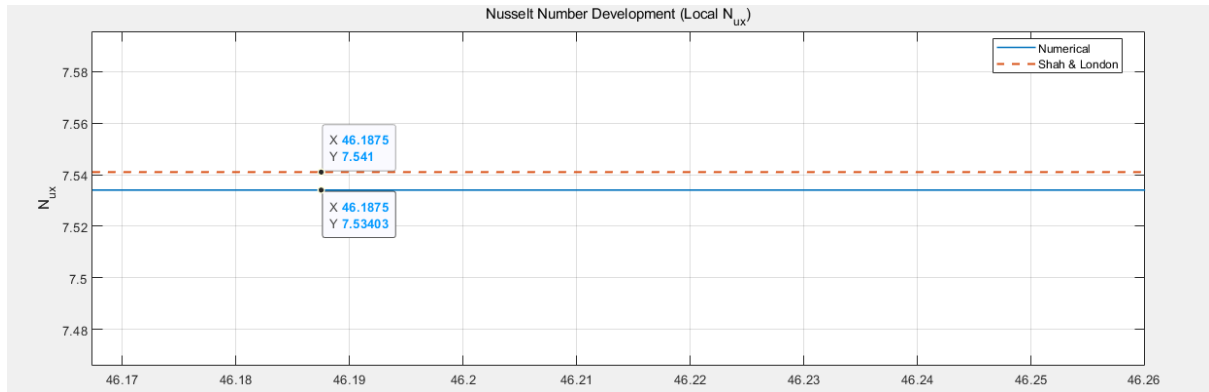
```

153 Gz = (Re * Pr * Dh) ./ x_cent;
154 Nu1 = 7.541;
155 Nu2 = 1.849 * Gz.^(1/3);
156
157 % 2) Shah & London local piecewise
158 Nu_SL_x = zeros(1,Nx);
159 for i = 1:Nx
160     if x_cent(i)==0
161         Nu_SL_x(i) = Nu_loc(1); % 입구부 한정 처리
162     elseif Gz(i) < 1000
163         Nu_SL_x(i) = Nu1 + 6.874*(Gz(i)/1000)^0.488 * exp(-245/Gz(i));
164     else
165         Nu_SL_x(i) = 2/3*Nu2(i) + 0.4;
166     end
167 end

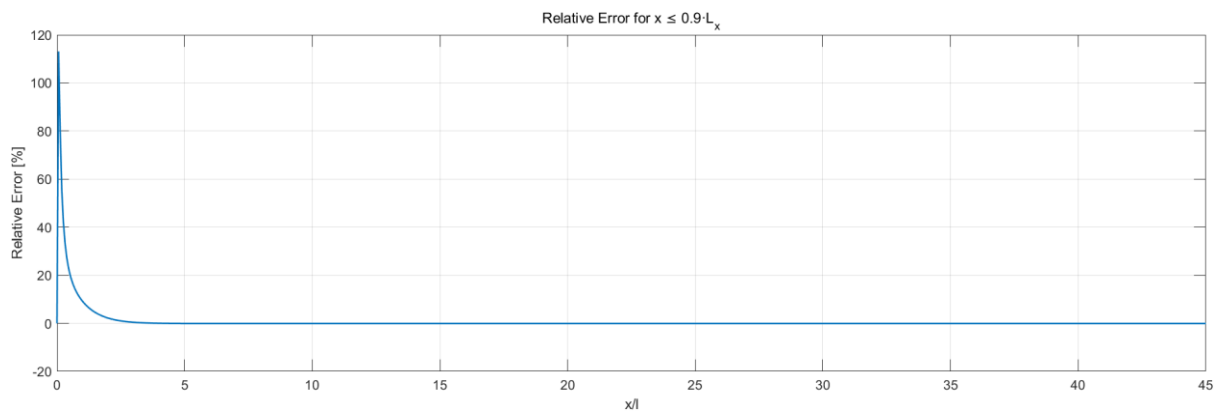
```

Plot을 해보면, 다음과 같다. 누셀넘버가 7.54에 가깝게 수렴 됨을 알 수 있었다.





논문에서 근거한 식으로는 7.541에 값이 수렴하고, 수치해석적으로는 7.534에 수렴한다. 이는 벽면쪽에 격자를 미세하게 배치하지 못했기 때문에, 기울기 계산에서 오차가 발생한 듯하다.



길이의 0.9에 해당하는 위치까지의 상대오차를 계산해보았다. 오차는 0.094% 정도 발생한다. 출구 쪽에서는 온도 변화가 매우 작기 때문에 약 1%의 오차가 발생하여, 0.9까지만 비교해보았다.

11일까지 Term Project 1을 모두 수행하는 코드 작성을 완료했다.

12일에는 제출용 레포트를 작성하면서 다시 한번 결과들을 뺐으면서 수정할 부분이 있는지 검토하려고 한다.

250512 : 레포트 작성 및 (d)번 thermal_fully_developed 위치 확인용 코드 추가

```

190 % Find index where Nu_loc becomes nearly constant (e.g., variation < 1%)
191 threshold = 0.0001; % 0.01%
192 for i = 3:Nx
193     if abs((Nu_loc(i) - Nu_loc(i-1))/Nu_loc(i-1)) < threshold ...
194         && abs((Nu_loc(i) - Nu_loc(i-2))/Nu_loc(i-2)) < threshold
195         x_fully_dev = x_cent(i); % 해당 위치에서 fully-developed
196         break;
197     end
198 end
199
200 fprintf('→ Fully-developed condition starts at x/l ≈ %.2f\n', x_fully_dev / l);
201

```